# Erica Mitchell – Research Engineer Technical Challenge

## Design

I chose to use C++ for the quantum circuit simulator so that the kernels (1- and 2-qubit operations) would be highly performant for many qubits. Also, C++ is currently my primary language, so I understand optimizations on the C++-level more deeply than for Python. Python was not completely disregarded though because plotting and circuit generation are much simpler if done using Python over C++. So, the repo I created, `ecm_qsim`, is a combination of the two; C++ for the simulation and Python for the tooling.

The layout is that of a standard C++ library as shown in the README.md:

```
include/ecm_qsim/   public headers
src/                library sources and the sim CLI (main.cpp)
tests/cpp/          C++ unit tests (Catch2)
tests/python/       Python tests (pytest)
python/ecm_qsim/    Python tooling: sim subprocess wrapper, idle-noise model and fit,
                    brickwork circuits
benchmarks/         brickwork circuit generator and benchmark runner
experiments/        experiment scripts and figures
examples/           example circuits
```

The core of the design centered around the different sections of the background in the prompt given: (1) state vector, (2) gates, (3) measurement, and (4) noise. Everything else is structure surrounding these components.

The state is held in the class `StateVector`, which consists of most of the memory used by the simulator. Any gate operation will run through its member functions, which can either be the reference implementation (`apply_1q`, `apply_2q`), the specialized kernel implementation (`apply_ry`, `apply_cnot`), or the gate fusion implementation (`apply_real_1q`, `apply_real_2q`). Measurement also requires the amplitudes stored as a complex Eigen vector by the `StateVector`.

All required gates were implemented as Eigen objects as well (complex dense matrices). To avoid forming large $2^n \times 2^n$ matrices, even the reference implementation utilizes simple qubit-wise operations so only the dense $2 \times 2$ and $4 \times 4$ matrices are allocated and computed. Gate correctness was easily verified using a variety of unit tests. Notably, the unitary and Hermitian properties could be checked along with the circuit identities in Nielsen & Chuang Ch. 4.

Measurement squares the stored amplitudes to get each outcome's probability. It places these probabilities end-to-end on [0, 1) and returns the outcome whose segment contains a random number `u`. Noise parameters (`p_gate`, `p_idle`, `p_readout`) are held in the `NoiseModel` struct. All random numbers come from one `std::mt19937_64` seeded from the command line. Random numbers in [0, 1) are made directly from its output bits, not with `std::uniform_real_distribution`, which can return exactly 1.0 on macOS and gives different values on different standard libraries.

## Validation

The noise experiment in the challenge was verified by deriving the analytic idle error and plotting the measured error versus this analytic error.

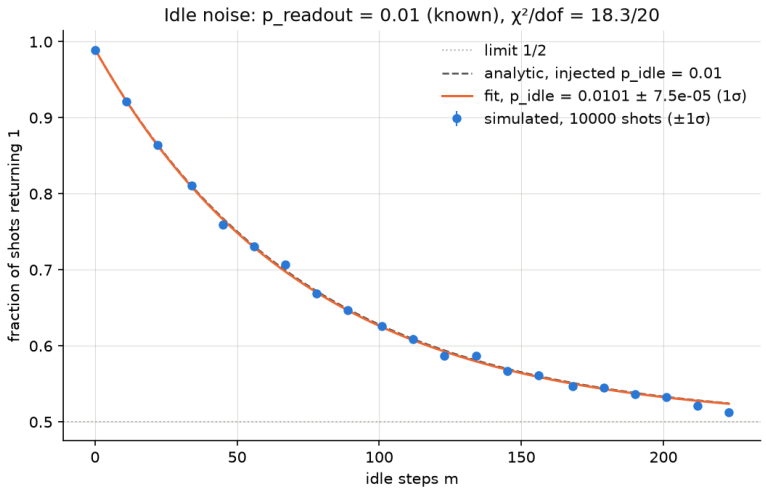



**Figure 1:** The idle error that occurs when a qubit in the 1 state is left idle for $m$ time steps. The analytic error is a dashed line while the fitted simulation error is in red. Error bars are present on the simulated data points but are so small at 10,000 shots that they are not visible on the graph.

The analytic idle error equation can be derived by understanding probability and the type of error that is occurring. There are bit-flip errors and phase-flip errors. Given the experiment, bit-flip errors are being detected, which for the Pauli gates gives a probability of

$$p_{\text{flip}} = \frac{2}{3} p_{\text{idle}}.$$

By casting this as a recurrence relation,

$$P_1(m+1) = \left(1 - p_{\text{flip}}\right)P_1(m) + p_{\text{flip}}\left(1 - P_1(m)\right) = p_{\text{flip}} + \left(1 - 2p_{\text{flip}}\right)P_1(m),$$

the probability of being in the 1 state is

$$P_1(m) = \frac{1}{2} + \frac{1}{2}\left(1 - \frac{4}{3} p_{\text{idle}}\right)^m.$$

I decided to treat $p_{\text{readout}}$ as a known since it would be classified as a measurement (or active) error not an idle error. The final probability of being in the 1 state, including readout error, is

$$f(m) = \frac{1}{2} + \frac{1}{2}(1 - 2p_{\text{readout}})\left(1 - \frac{4}{3}p_{\text{idle}}\right)^m.$$

**Table 1:** Shot count justification ($p_{\text{idle}} = 0.01$, $p_{\text{readout}} = 0.01$, $m = 0 - 223$, 21 points).

| Shots per $m$ | Std. Error of $p_{\text{idle}}$ | Relative Error |
|---|---|---|
| $10^3$ | $2.34 \times 10^{-4}$ | 2.3% |
| $10^4$ | $7.39 \times 10^{-5}$ | 0.74% |
| $10^5$ | $2.34 \times 10^{-5}$ | 0.23% |

The shot count was tested for a variety of shots, Tab. 1. From the data, it is observed that under 1% relative error is achieved with 10,000 shots.

**Table 2:** Failure at very small $p_{\text{idle}}$ ($p_{\text{readout}} = 0.01$, $10^4$ shots per $m$).

| Injected $p_{\text{idle}}$ | Relative Error ($m \leq 220$) |
|---|---|
| $10^{-2}$ | 0.7% |
| $10^{-3}$ | 1.0% |
| $10^{-4}$ | 3.8% |
| $10^{-5}$ | 27% |
| $10^{-6}$ | 260% |

The analytic formula for the idle error fails for both very small and very large $p_{\text{idle}}$. For very small $p_{\text{idle}}$, the estimate loses precision, not accuracy (Table 2). With $m$ fixed at $m \leq 220$ and $(4/3)\, p_{\text{idle}} \cdot m \ll 1$ over the whole range, the curve is a straight line with slope $-\frac{2}{3}(1 - 2p_{\text{readout}})\, p_{\text{idle}}$, and the fit measures only that slope. Because p_readout is known, the uncertainty on the slope does not depend on $p_{\text{idle}}$. It is set by the shot noise near $f \approx 1 - p_{\text{readout}}$ and by the $m$ grid. This gives a floor of $\sigma(p_{\text{idle}}) \approx 2.6 \times 10^{-6}$, so the relative error grows as $1/p_{\text{idle}}$. Below about $3 \times 10^{-6}$ the estimate cannot be told apart from 0. The fix is to scale the $m$ range with $1/p_{\text{idle}}$, as the default grid does (three decay lengths). More shots help only as $1/\sqrt{N}$.

For very large $p_{\text{idle}}$, the estimate stays accurate (Table 3), but the experiment no longer tests the model. With $r = 1 - \frac{4}{3}p_{\text{idle}}$, the signal above $\frac{1}{2}$ is $\frac{1}{2}(1 - 2p_{\text{readout}})r^m$. It

falls below the shot noise after about $\frac{ln(0.01)}{ln|r|}$ steps: 9 steps at $p_{\text{idle}} = 0.3$ and 4 at $p_{\text{idle}} = 0.5$. The $m = 0$ point never carries information, because $f(0) = 1 - p_{\text{readout}}$ for any $p_{\text{idle}}$. At $p_{\text{idle}} = \frac{3}{4}$, $r = 0$ and $f(m) = \frac{1}{2}$ for every $m \geq 1$, so the fit reduces to solving $f(1) = \frac{1}{2} + \frac{1}{2}(1 - 2p_{\text{readout}})(1 - \frac{4}{3} p_{\text{idle}})$ for $p_{\text{idle}}$. One point gives a precise number, but it cannot show that the decay is exponential. For $p_{\text{idle}} > \frac{3}{4}$, $r < 0$ and the points alternate about $\frac{1}{2}$, so the odd $m$ fix the sign of $r$.

**Table 3:** Failure at very large $p_{\text{idle}}$ ($p_{\text{readout}} = 0.01$, $10^4$ shots per $m$, $m = 0 - 10$).

| Injected $p_{\text{idle}}$ | $r = 1 - \frac{4}{3}p_{\text{idle}}$ | Relative Error |
|---|---|---|
| 0.3 | +0.600 | 1.0% |
| 0.5 | +0.333 | 1.2% |
| 0.75 | 0.000 | 1.0% |
| 0.9 | −0.200 | 0.77% |
| 1.0 | −0.333 | 0.59% |

# Optimizations

**Hardware and build.** Apple M3 (8 physical cores), 24 GiB memory, macOS 26.6.2, no GPU. Built as Release with Apple clang 21.0.0 and -O3 -DNDEBUG. The theoretical peak memory bandwidth of the M3 is 100 GB/s.

From the papers referenced, I knew there were three optimizations I really wanted to tackle: (1) specialized kernels for gate application, (2) naïve gate fusion, and (3) memory reduction. The reference showed clearly that most of the time is spent in gate application, so kernel optimization or gate fusion are essential for computational speed. The papers also note that for their more sophisticated simulators, memory becomes the major bottleneck, which made me want to try some memory optimization.

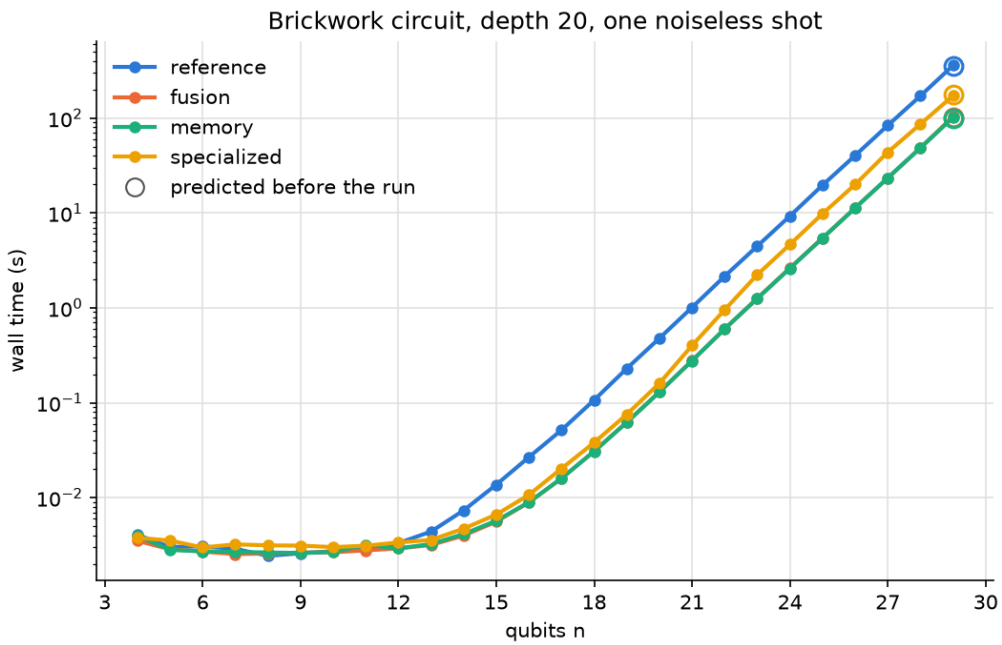



**Figure 2:** Performance of the brickwork circuit for many qubits ($n = 4 - 29$).

## Specialized Kernels

In the study by Häner and Steiger, they improve single-core performance of their gate kernels. I took a similar approach and looked to improve the RY(θ) and CNOT gate kernels by avoiding direct application of the dense matrices and only applying the gates to their relevant amplitudes.

The new `apply_ry` multiplies each amplitude pair by the real rotation coefficients directly. This reduces the cost from 14 to 6 real FLOPs per amplitude. It also avoids the `continue` branch in `apply_1q`. The outer loop steps through blocks of $2^{k+1}$ indices. In each block the first $2^k$ indices have bit $k$ clear, and the inner loop visits only those, pairing each index $i$ with $i + 2^k$. The result is $2^{n-1}$ iterations instead of $2^n$. The inner loop has no branch and unit stride, and Clang vectorizes it.

The new `apply_cnot` replaces the matrix multiplication with swaps, so it uses no FLOPs. It changes only the amplitudes with the control bit set: each one is swapped with its partner, which differs only in the target bit. This reads and writes half of the state, where the reference `apply_2q` reads and writes all of it. Three nested loops visit exactly the $2^{n-2}$ indices with both gate bits clear, so no iterations are skipped; `apply_2q` visits all $2^n$ indices and skips three-quarters of them. Clang does not vectorize the inner loop of `apply_cnot`.

For higher qubit counts ($n > 12$) with the many qubits benchmark, a nearly double or more speed-up is observed using these specialized kernels. For the many shots benchmark, an about $3.5\times$ speed-up is observed!

## Gate Fusion

In the simulator QuEST and in the study by Huang et al. (qfusion-opt), gate fusion combines consecutive gates into one matrix, so one pass over the state vector applies several gates. Here, I fuse the gates so that no fused matrix acts on more than two qubits. Fusion reduces the number of passes over the state, but it adds FLOPs.

The new `fuse_trajectory` puts the circuit gates and the sampled Pauli errors in program order and merges them greedily. It keeps a set of open clusters on disjoint qubits. Each new operation is multiplied into the clusters it touches, as long as the result acts on at most two qubits. If not, those clusters are emitted and the operation starts a new cluster. The only operations that change order are ones on disjoint qubits, and those commute. For the brickwork circuit with depth 20 this turns $20n$ RY gates and $10(n-1)$ CNOTs into $10n-1$ fused gates: 289 instead of 860 at $n=29$.

X, Z, H, RY, CNOT and CZ are all real, so most clusters stay real. Two new kernels, `apply_real_1q` and `apply_real_2q`, use the same loops as `apply_ry` and `apply_cnot`. Each real coefficient multiplies the real and imaginary parts of the amplitudes directly. A real $4\times4$ matrix costs 14 FLOPs per amplitude, against 6 for RY and 0 for CNOT. So, at $n=29$ the fused circuit does about 14% more arithmetic (3974 vs. 3480 FLOPs per amplitude). Similar to `apply_cnot`, Clang does not vectorize the inner loop of `apply_real_2q`.

For the many qubits benchmark, fusion gives a further $1.2\times$ speed-up over the specialized kernels at $n=16-20$. For $n\geq24$ it gives about $1.7-1.8\times$, which is about $3.5\times$ faster than the reference. That is less than the $3.0\times$ the drop in passes predicted. The unvectorized $4\times4$ kernel is limited by its instructions, not by memory bandwidth: it reaches only 48% of peak bandwidth. For the many shots benchmark the gain is smaller, $1.07\times$ over the specialized kernels ($3.77\times$ over the reference), because building the fused matrices for each shot takes 8% of the run time.

## Memory Reduction

After fusion, the peak memory at $n=29$ was 12.01 GiB, which is 24 bytes per amplitude. 16 bytes are the complex state vector. The other 8 bytes are a copy of the probabilities, $|\text{amplitude}|^2$. That copy isn't needed, instead $|\text{amplitude}|^2$ can be computed each time it reads an entry.

A new overload samples the state in-place. The existing algorithm moved unchanged into one function template that reads any Eigen expression of double. The vector overload passes its vector, and the state overload passes the lazy expression `amplitudes_.cwiseAbs2()`.

For the many qubits benchmark, the peak memory at $n = 29$ falls from 12.01 GiB to 8.01 GiB, a one-third reduction. The peak now matches the size of the state vector to within 0.2% for $n \geq 26$. The wall time (101.5 s vs. 103.7 s) and the many shots throughput (3752 vs. 3678 shots per second) changed by about 2%, which is within run-to-run noise.

## Future Optimizations

If given more time, the profiling shows that most of the time in the simulator is now spent in the unvectorized $4 \times 4$ kernel. Vectorizing this kernel would be top priority to increase performance.

I'm also interested in looking at gate scheduling paired with gate fusion for multinode computations would be interesting to investigate the referenced papers nearly all mention gate scheduling. With HPC resources, it would be interesting to profile and optimize the gate fusion so that the operations are load balanced across nodes and scheduled effectively to avoid stalling threads and ranks. As the number of qubits and depth of the circuit increase, careful orchestration of the gate operations and data movement will become even more important to simulate quantum circuits accurately.